User-friendly ESP32 firmware application to simplify network configuration for open-source IoT devices

Meng Xian Yang
& Federico Dallo

ISP-CNR – Feng Chia University Students' Internship – 29 April 2026

The problem – Needs

- Necessary to build new tools (models and data) to estimate
  - **Indoor exposure** to air pollution
  - Main **causes** (activities, building factors, infiltration, etc)
- The results will be used
  - to inform target stakeholders (municipalities, building sector, environmental agencies, ...)
  - to develop specific intervention (e.g., new ERF, greening, enhance air filtration in buildings, ...)

→ EU funded project **healthRiskADAPT**

WP leader
ISP-CNR

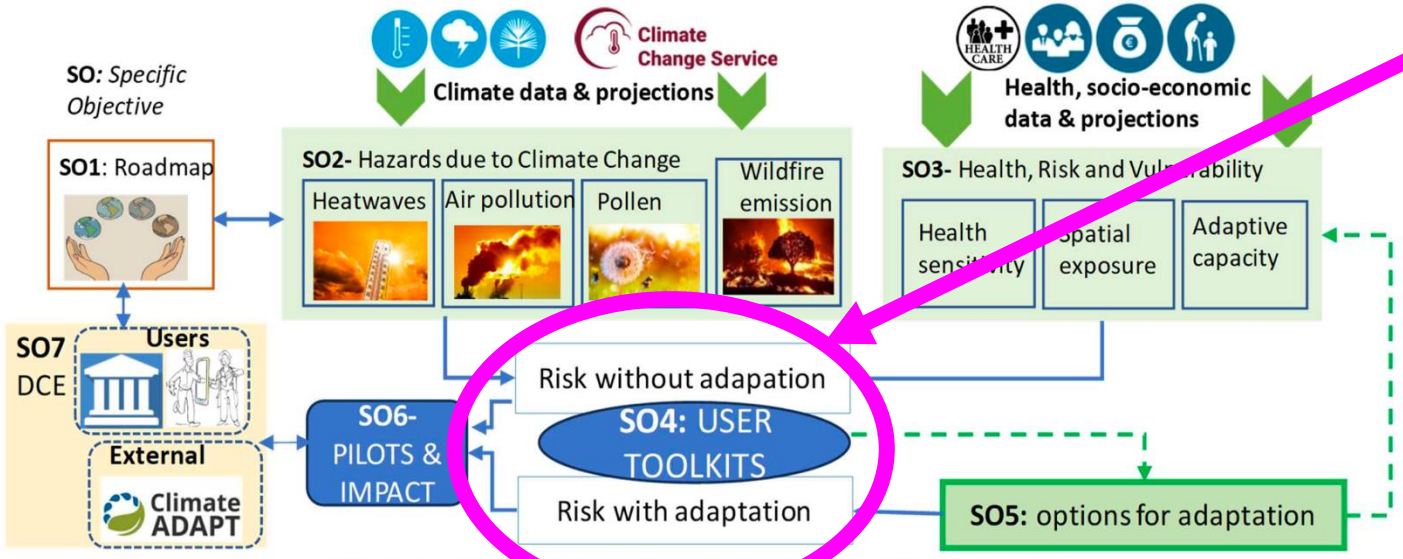
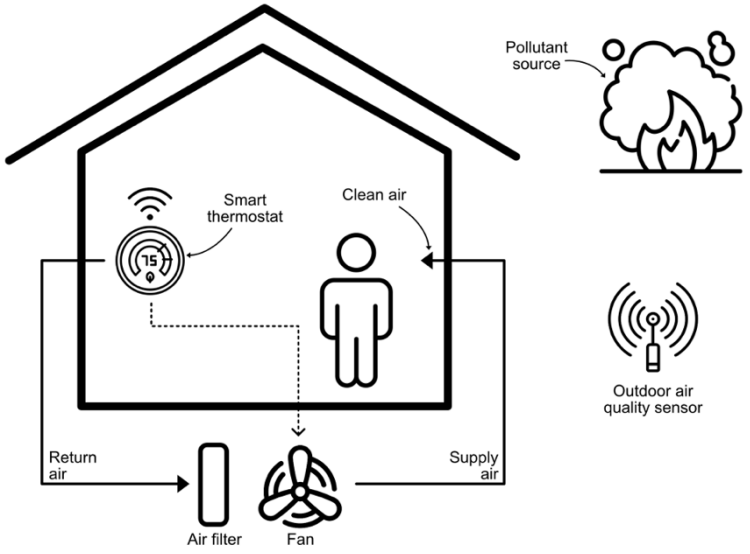
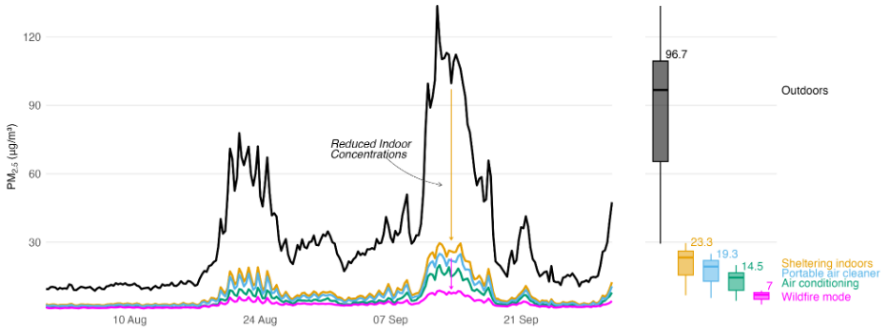


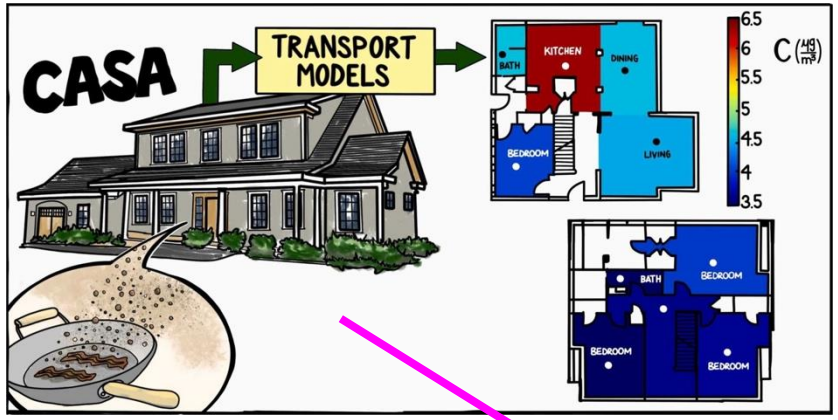
Fig. 1.1 *riskADAPT's schematic and its specific objectives*

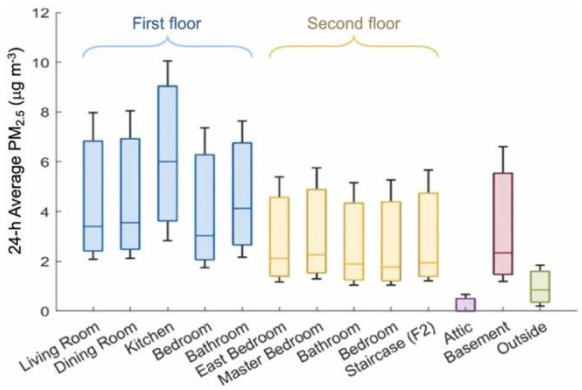
The problem – Existing Solution





Dallo, Federico, et al. "Using smart thermostats to reduce indoor exposure to wildfire fine particulate matter (PM2. 5)." Indoor Environments (2025): 100088.



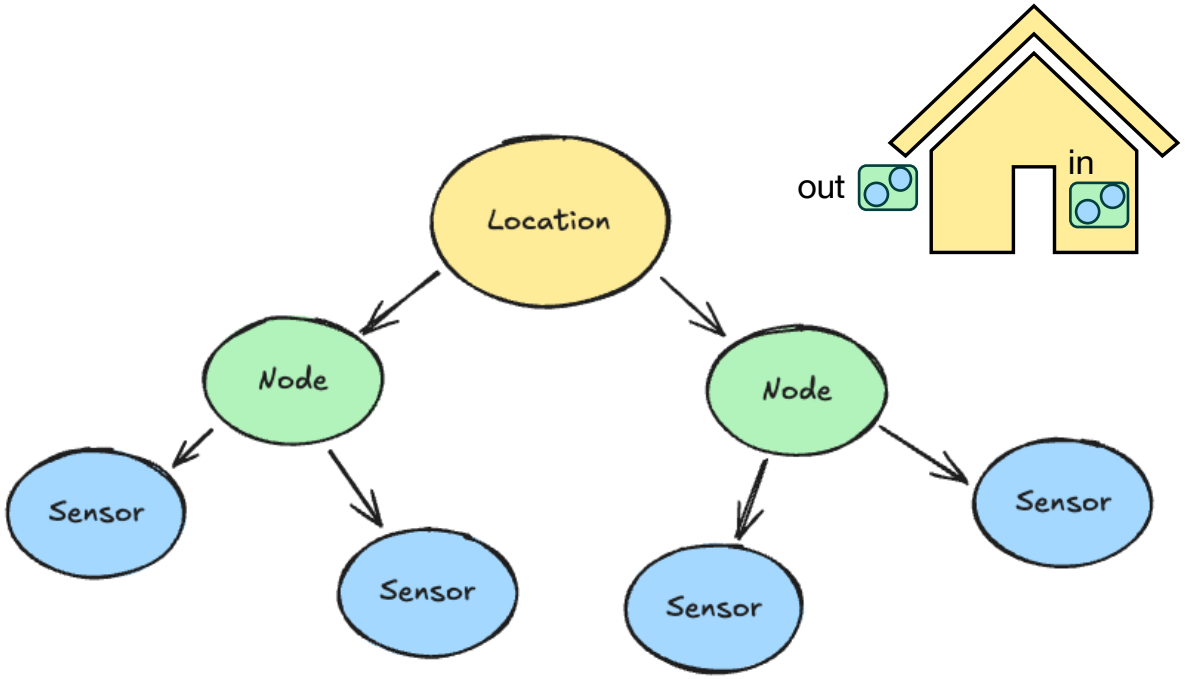


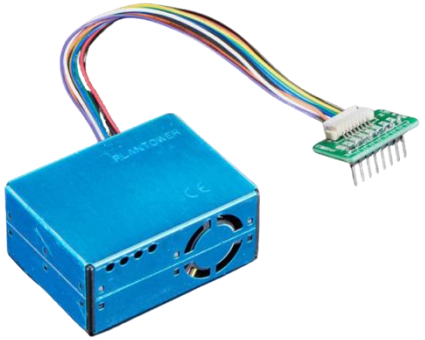
Martin, Andrew B., et al. "Investigating transport of particulate matter from cooking emissions in a multi-story house using low-cost sensor measurements and different modeling approaches." Indoor Environments (2025): 100126.

e.g., using Purple Air sensors

Locations & Sensors Management

- **Locations**
  - Represent physical places (e.g. home, school, park)
  - Group multiple devices under one area
  - Store GPS coordinates, address, and metadata
- **Nodes (Devices)**
  - Each *Node* = one physical device (ESP8266 or similar)
  - Connects to Wi-Fi and sends data to the platform
  - Identified by a unique registration code
- **Sensors**
  - The actual measurement components attached to a node
  - Examples: PM2.5, PM10, temperature, humidity, pressure
  - Each sensor produces time-series data
  - Easily configurable and replaceable





Recap for Users! *https://github.com/theRosyProject/public_ParticularMatter

Web App

Create User

Location 1

Node 1

...

Node *n*

One Time Key

*Application Output

To the Dashboard

Build Device

Install Firmware

Power Up

Access Dev. WiFi

One Time Key

*User Input

Measurements

Database

Credential Check and Device Registration

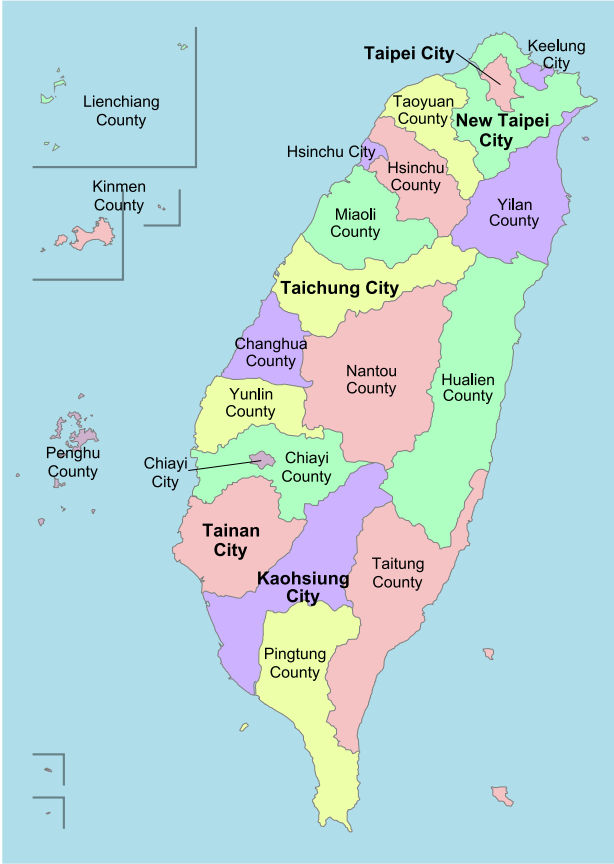
User-friendly ESP32 firmware application to simplify network configuration for open-source IoT devices

Meng Xian Yang
& Federico Dallo

ISP-CNR – Feng Chia University Students' Internship – 29 April 2026

Introduction – Meng Xian Yang (Vincent)

- Senior CS Student @ Feng Chia University, Taiwan
- Hometown: Tainan (The Food Capital of Taiwan)
- Interests: Badminton, Tennis, Exploring New Technologies



My Tech Stack & Background

- 3 Years of Software Development | 1 Year of AI Engineering
- Software Engineering
  - **Focus:** Full-stack Development · System Integration · Cloud Deployment
  - **Tech Stack:** Node.js, NestJS, React, PostgreSQL, Firebase, Docker, GCP, etc.

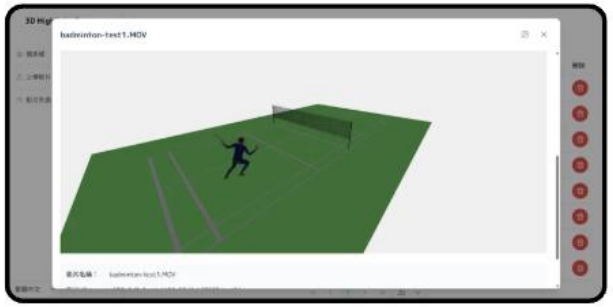
- AI & Computer Vision
  - **Focus:** Object Detection · Multi-Object Tracking · Model Training
  - YOLO, SAM, DeepSORT, etc.

Earlier Work: 360° Badminton Highlight System

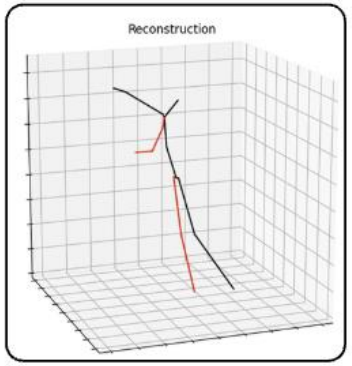
Consiglio Nazionale delle Ricerche

ISP Istituto di Scienze Polari
Consiglio Nazionale delle Ricerche

System Overview Screen

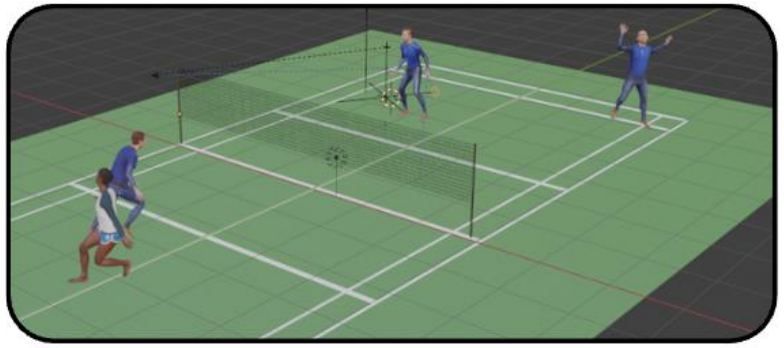


Pose Estimation & Skeleton Construction



Human Detection & Tracking

Model Render

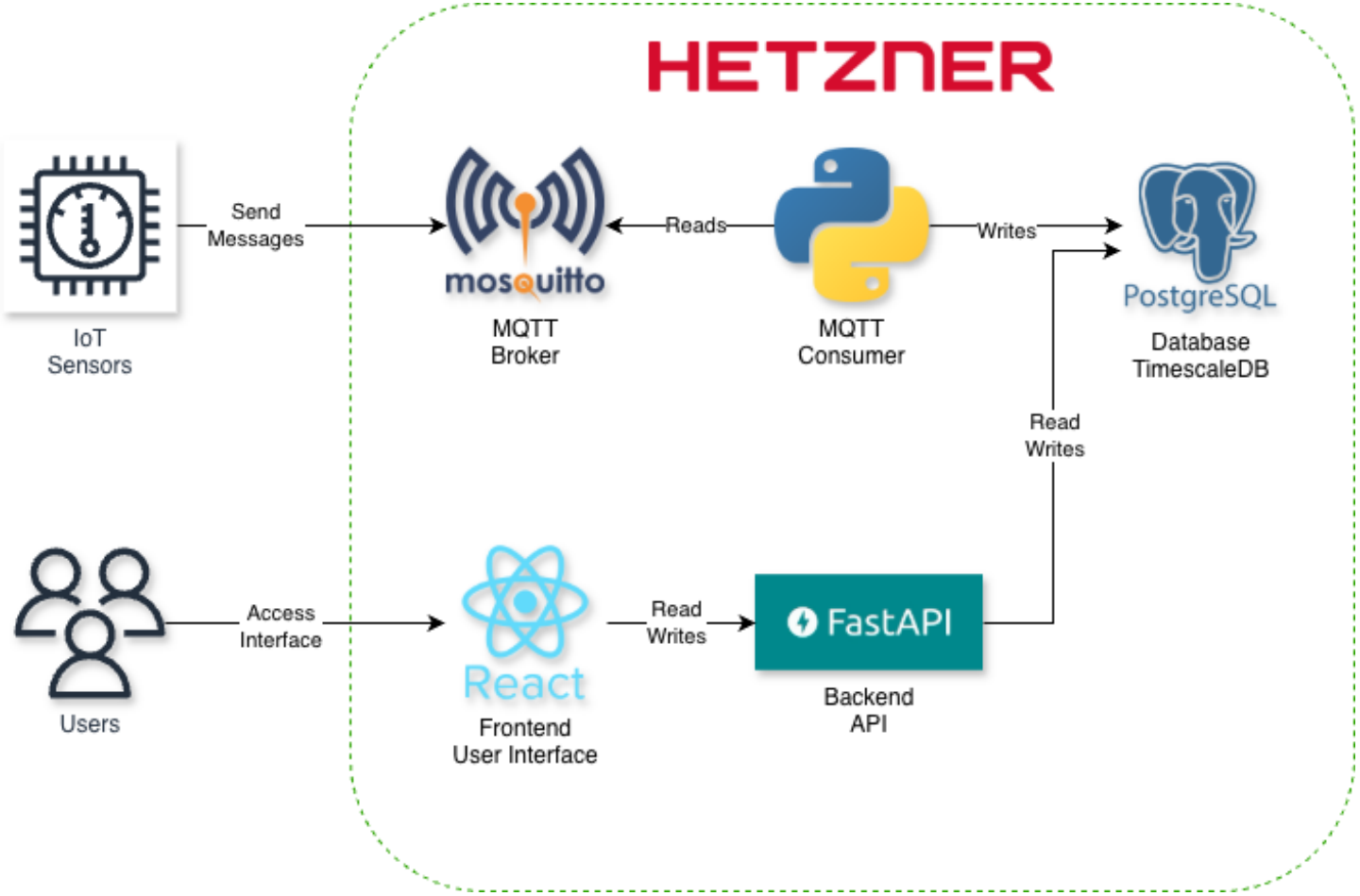


Consiglio Nazionale delle Ricerche

ISP Istituto di Scienze Polari
Consiglio Nazionale delle Ricerche

CNR: World-Class Research in a Historic City

Particular Matter Framework



System Architecture Overview

The solution – what?

Device/s

Indoor/Outdoor

Indoor

MVP
- Device Firmware ✓

Next Steps
- Commercial Device Integration trough HomeAssistant (~6 months)
- Integration with Industry Partner Building Automation System (~1 yr)

Device GUI – WiFi AP

"One-time" GUI ✓
- Device Configuration
- Local Raw-Data Visualization (debug)

User Interface/s

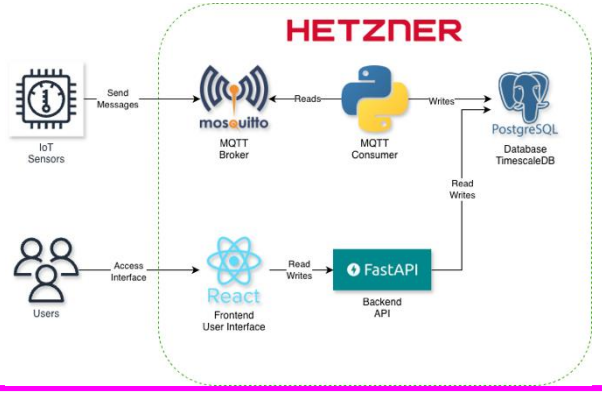
Web App

User Application ✓
- User Registration
- Sensor Networks Configuration
- Data Visualization / Download

Next Steps
- Improve Design

data

Data Management



Core Application ✓
- Device Registration
- Data Handling
- Backend

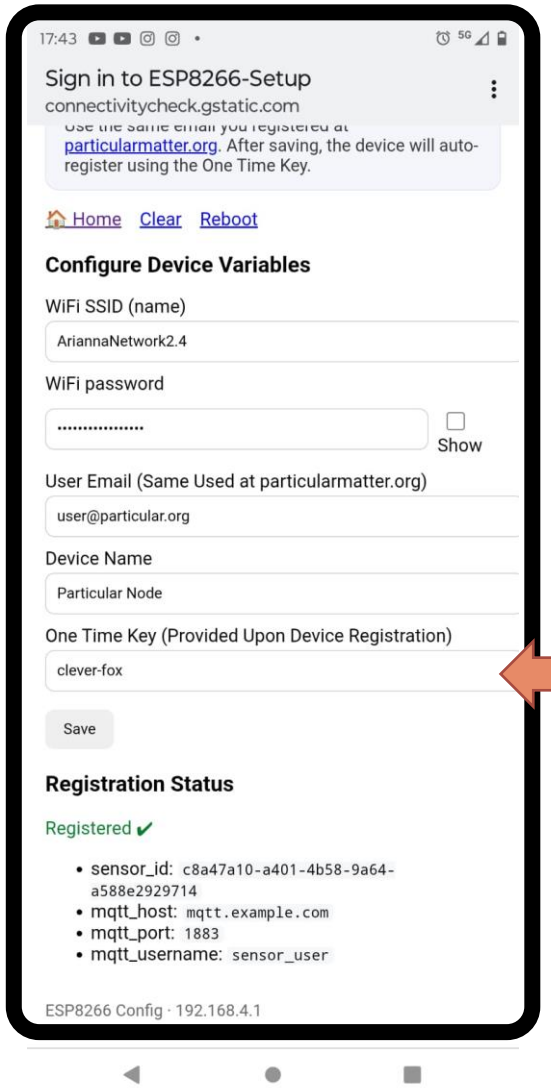
Next Steps
- Mass Balance Model Integration

DIY Device – Registration

1) Plug the device to connect to the WiFi Access Point:

SSID: ParticularMatterDevice-Setup
PWD: particularmatter

Automatic Redirect (e.g., like airports' captive portals)



The One Time Key is generated in the Web App by the User

(next slide)

WebApp – Sensor Networks Configuration

Web App

Create User

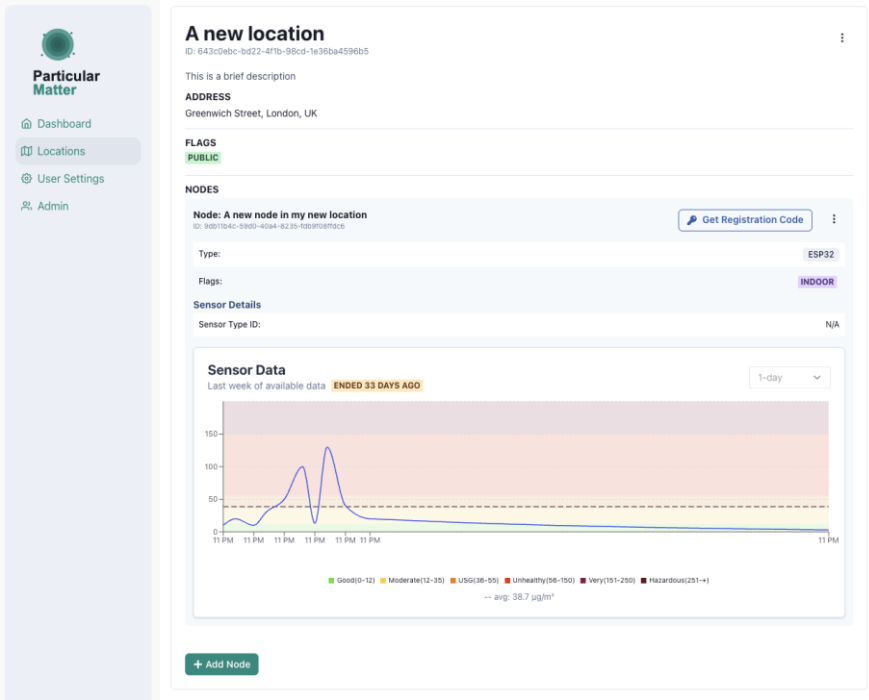
Location 1

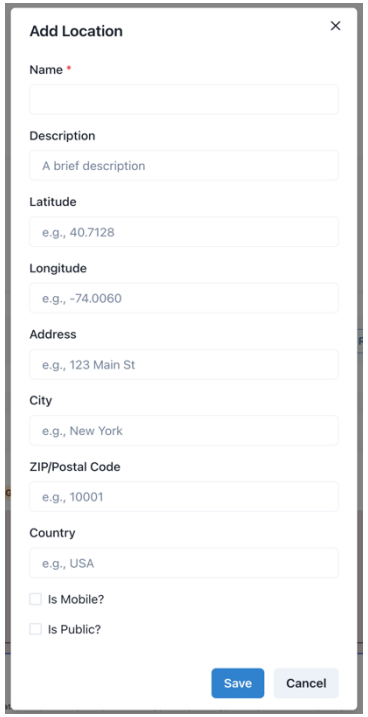
Node 1

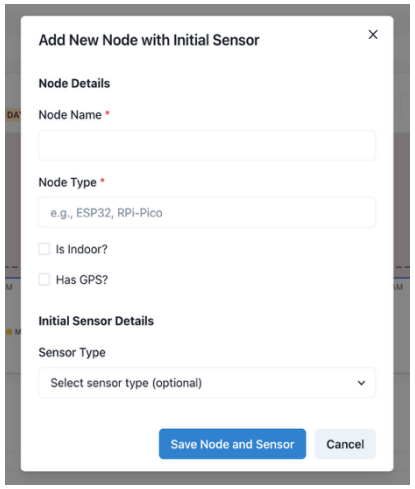
One Time Key

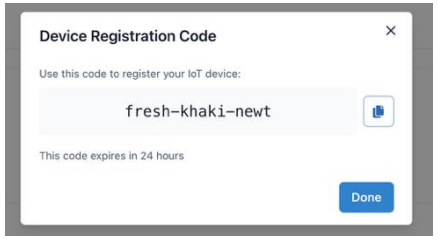
...

Node *n*









From Prototype to Open Source

Current Bottlenecks:

- **Software:** Monolithic design (hardcoded in `main.cpp`) & steep learning curve.
- **Hardware:** Legacy ESP8266 limits performance and lacks pins for future expansions.

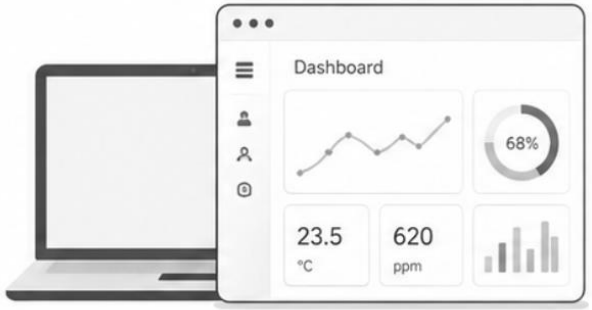
The Upgrade Vision (healthRiskADAPT):

- **Software Open-Source:** Build a modular, user-friendly repository for non-CS researchers.
- **Hardware Migration:** Upgrade to the **ESP32** microcontroller.
- **Scalable Architecture:** Redesign the wiring and power distribution to support future sensors (e.g., GPS, Temp., Relative Humidity, Pressure, CO_2, CH_4, ...).

The Solution Outline

- System Modularization
- Frontend-Backend Decoupling
- API-First Development
- CI/CD Pipeline (Continues Integration / Continues Deployment)

New Firmware Architecture



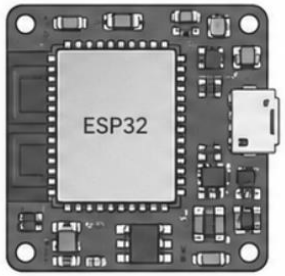
Client-Side (User Interface)

- Driven by **Vue 3** & **Vite.**
- Built with **TypeScript.**
- State managed by **Pinia.**
- Decoupled testing via **MSW.**
- Clean code ensured by **Husky** pre-commit hooks.
- Served via **LittleFS.**

..any browser

REST API →

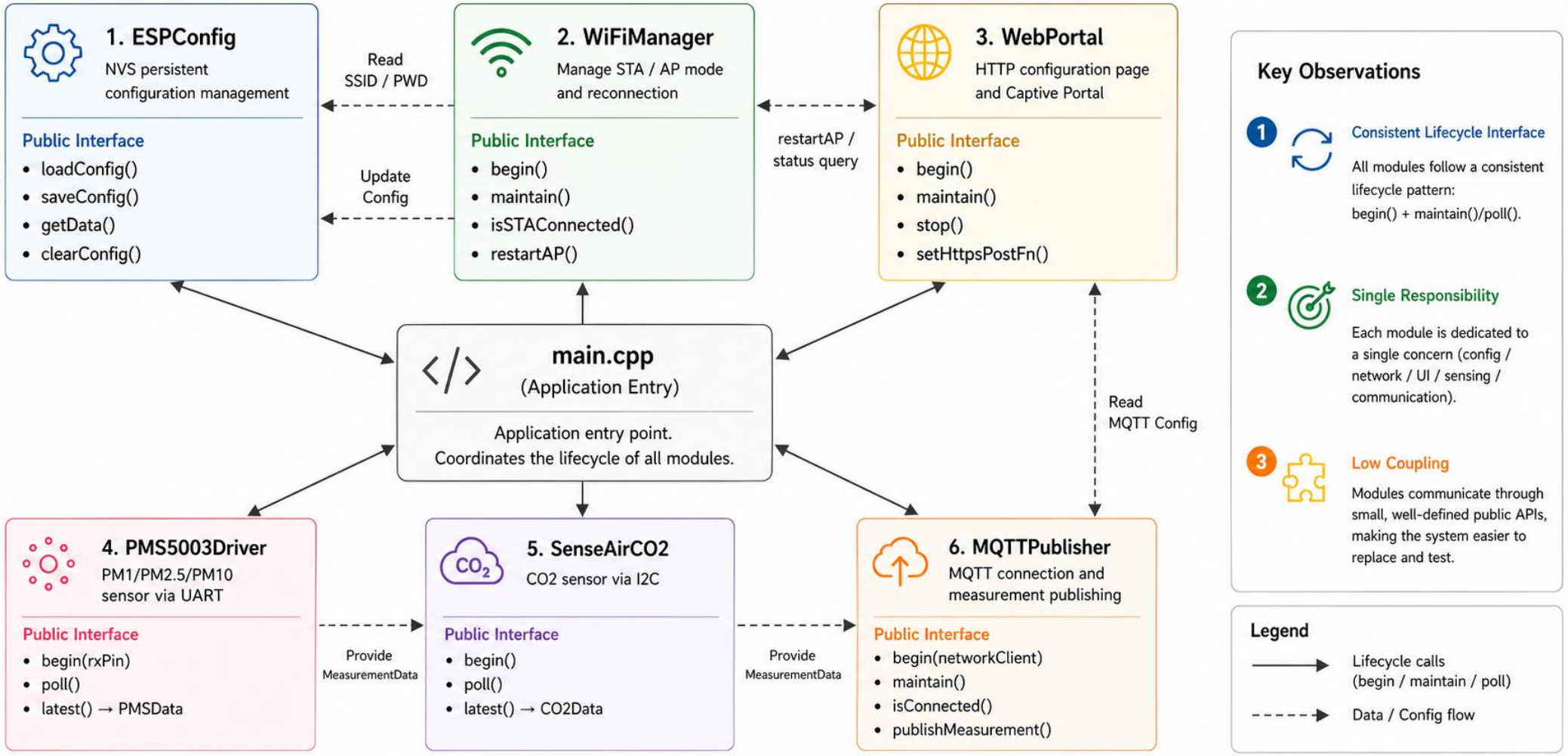
← JSON Payloads



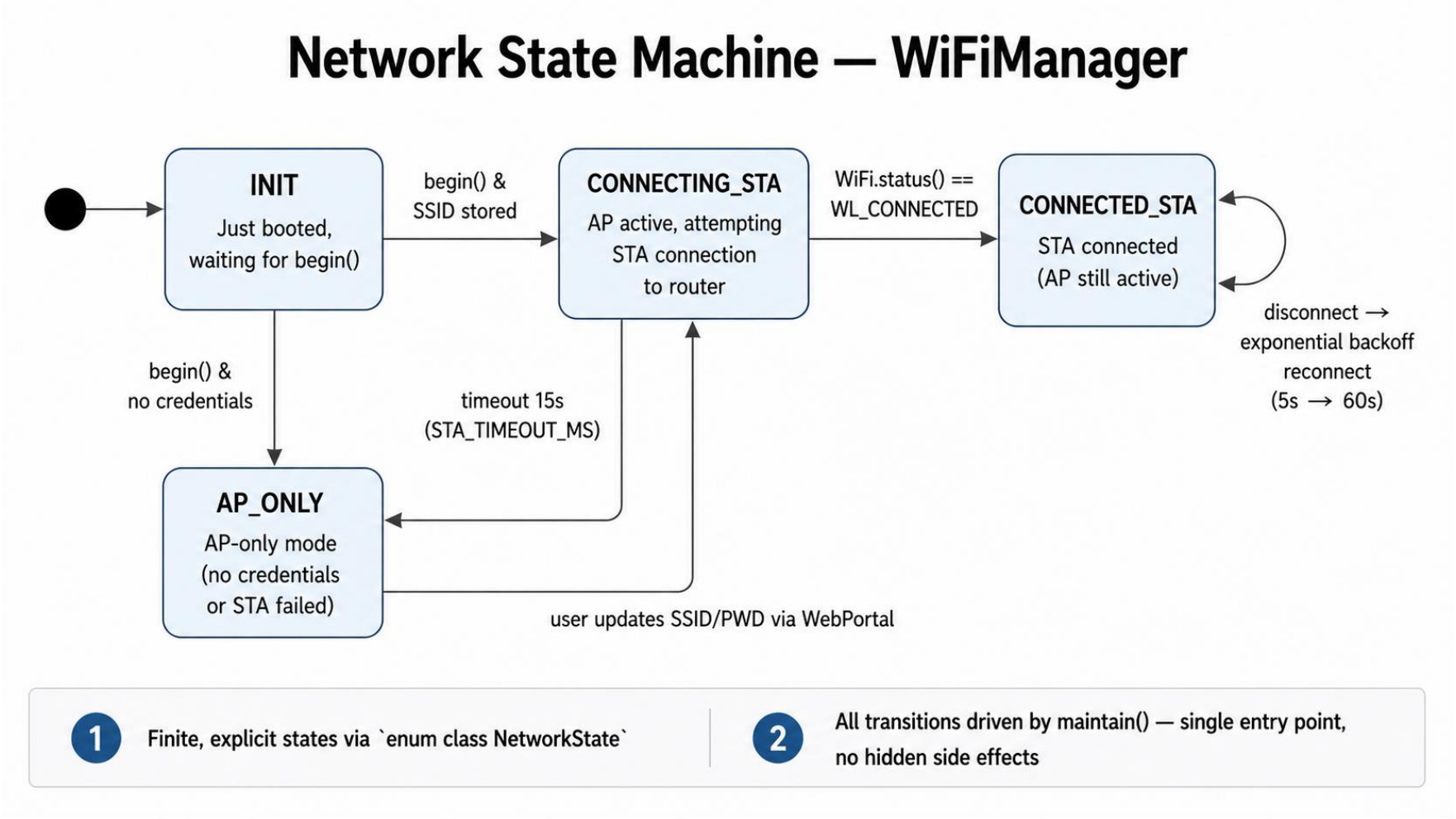
Server-Side (Embedded Core)

- Robust Embedded **C++** firmware on **ESP32**.
- Provides standard **REST API** endpoints.
- Modular libraries for Sensors (PM2.5, CO2).
- Modular Network & Portal logics.
- Integrated **MQTT** logic for external data.
- Uses **LittleFS** file system.

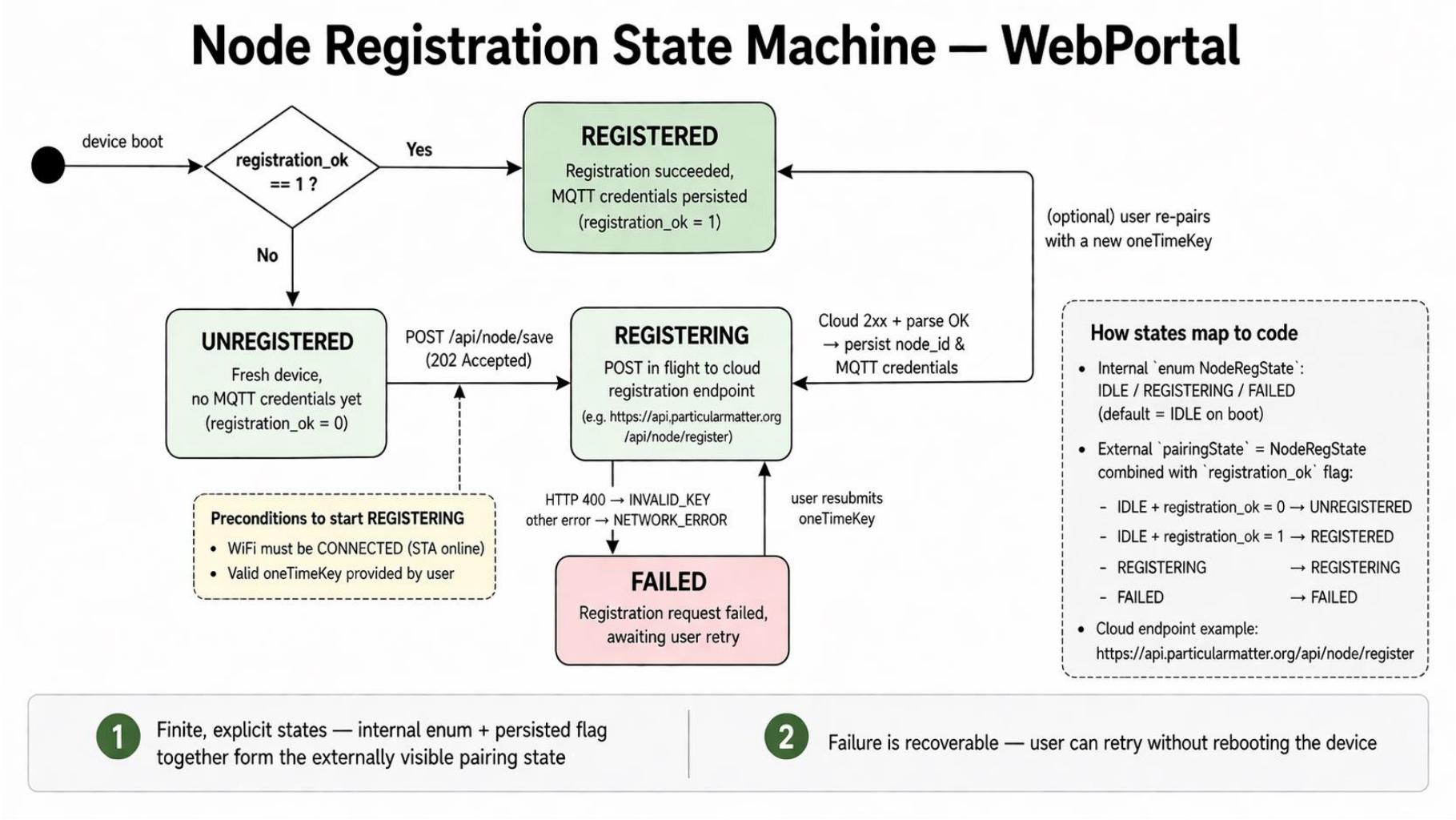
System Modularization



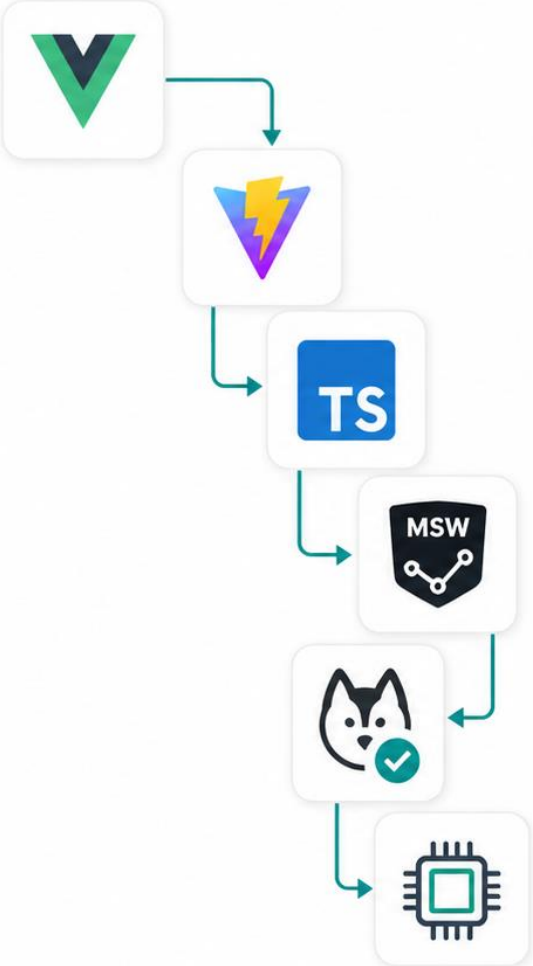
The Core State Machine



The Core State Machine



Frontend-Backend Decoupling



Tech Stack

- Built on **Vue 3** + **Vite**, state managed by **Pinia**
- Fully typed with **TypeScript** for safer refactoring & better DX

Separation of Concerns

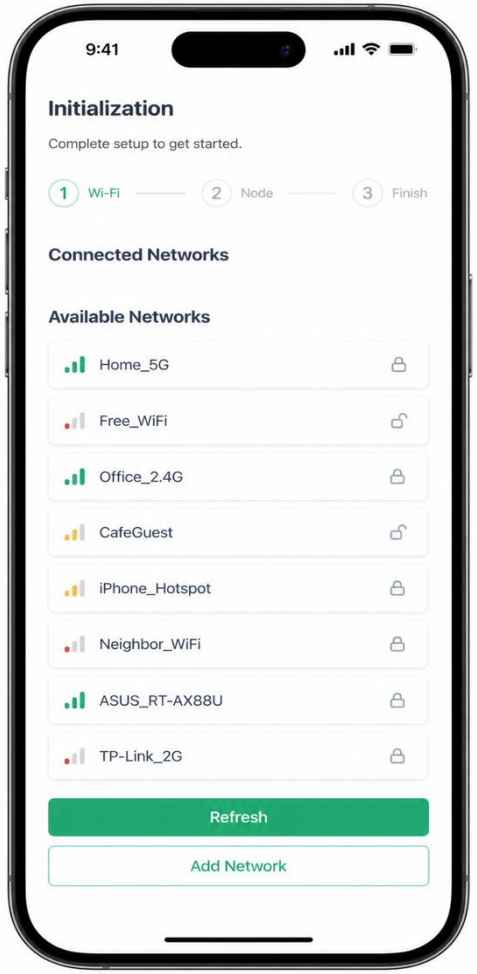
- UI logic fully decoupled from the C++ firmware core
- Frontend & firmware evolve **in parallel**, reducing backend overhead

Quality & Scalability

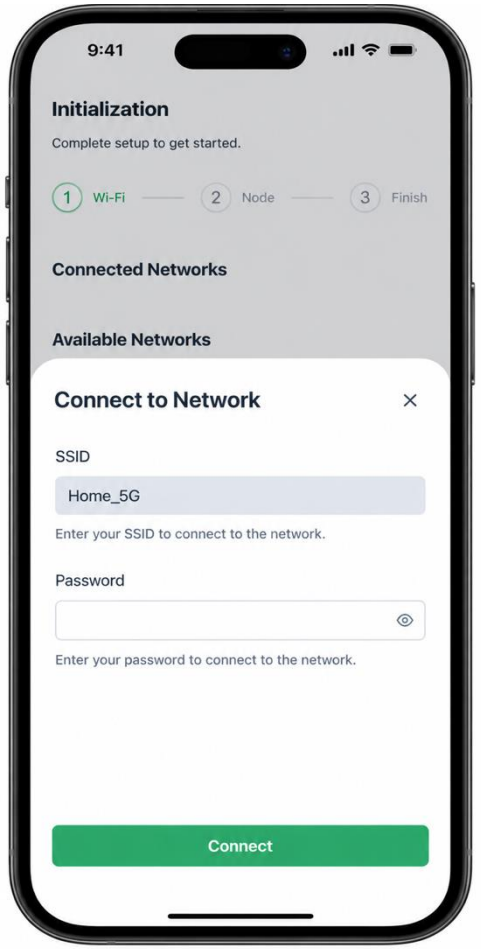
- **MSW** (Mock Service Worker) — develop & test UI without physical ESP32
- **Husky** + **ESLint** — automated pre-commit gates keep the codebase clean
- Deployment — bundled static assets served directly from **LittleFS** on ESP32

Initialization

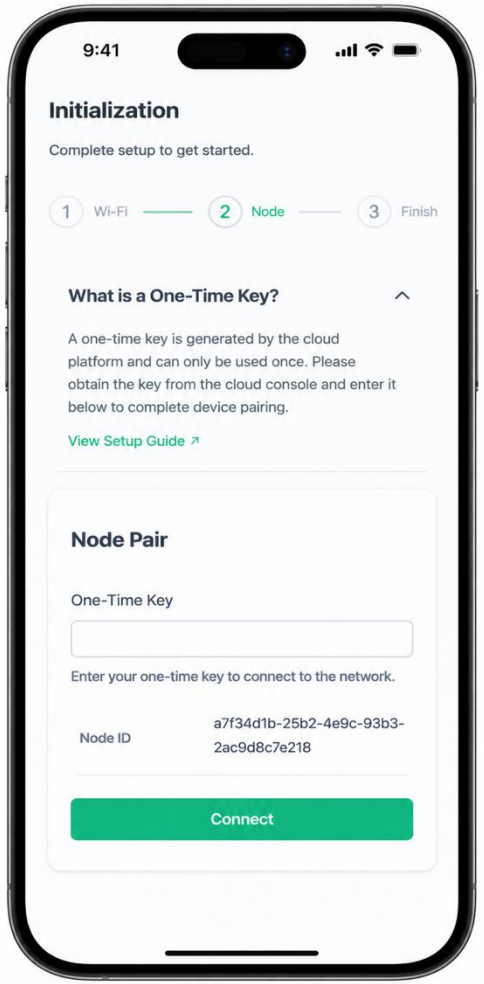
1.Searching Internet



2.Typing Password of WiFi

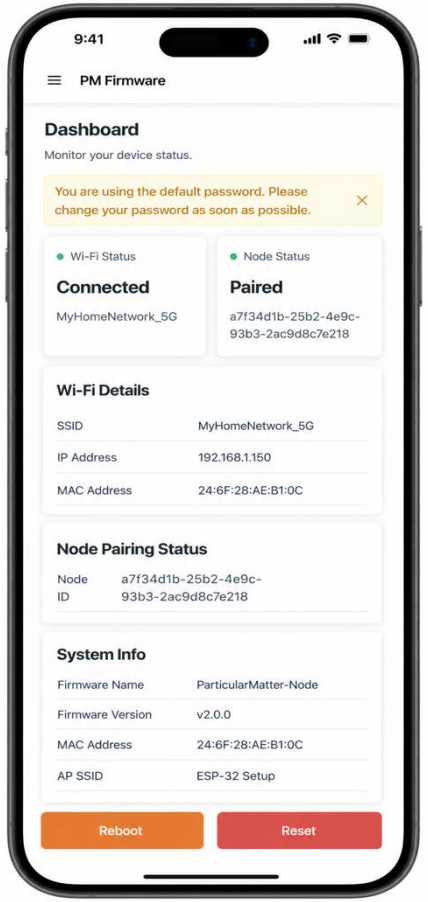


3.Pairing The Node

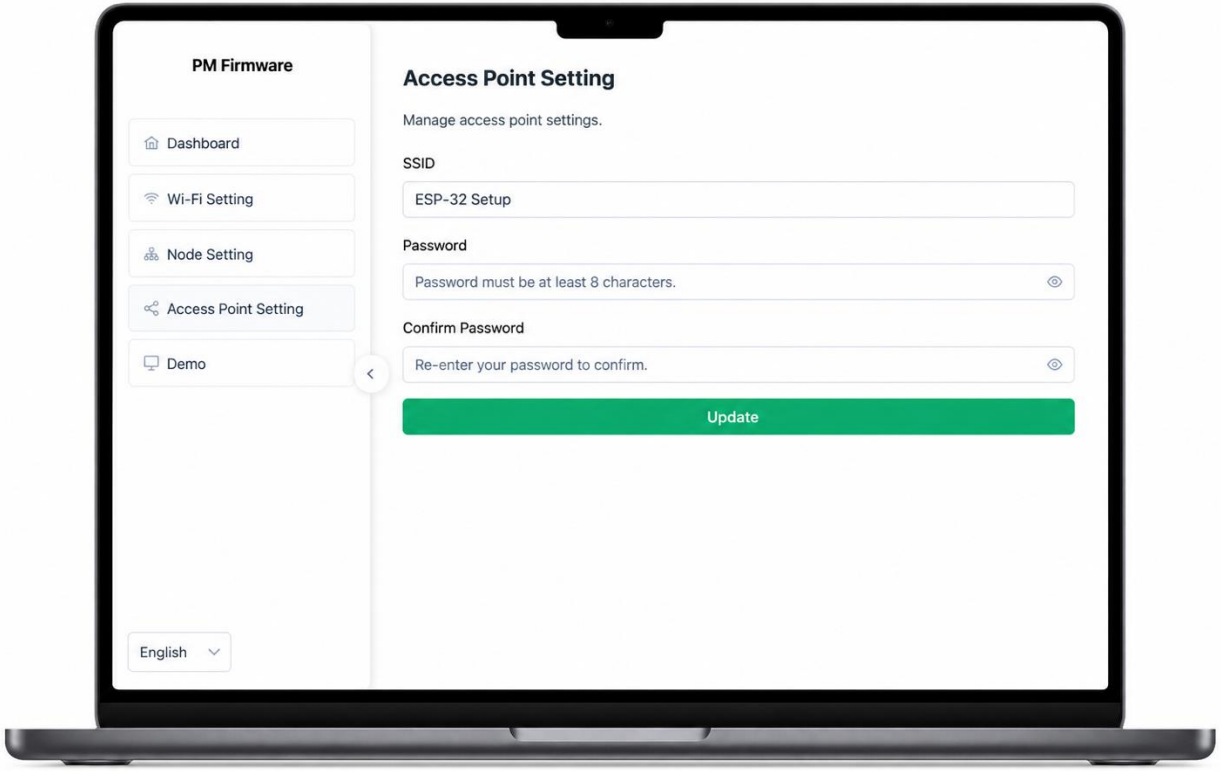


Real-Time Monitoring

Dashboard

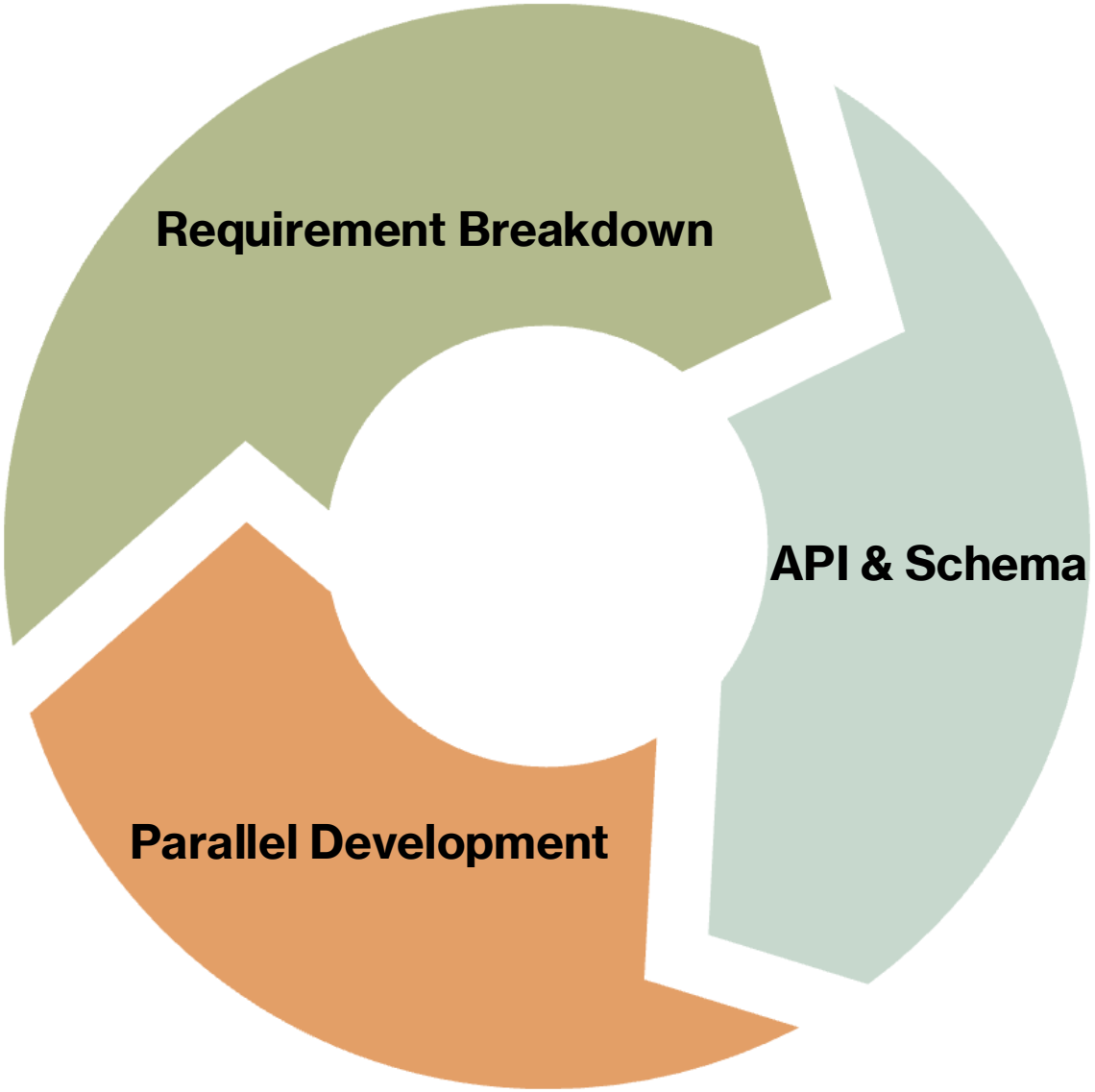


AP Setup

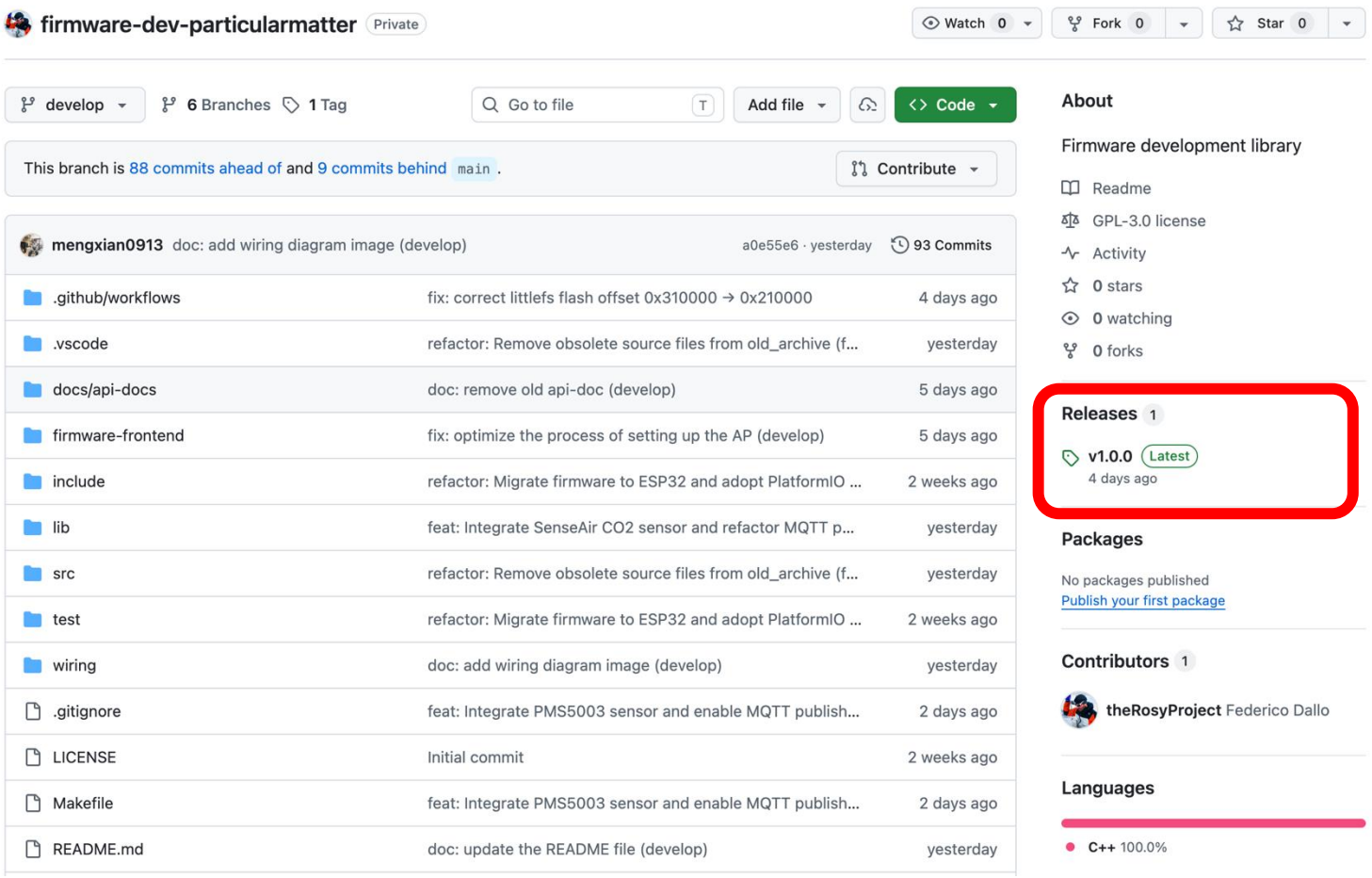


The API-First Cycle

When there is a new requirment comes out!



CI/CD & Automated Pipeline



What is next step ?

1. Hardware Ecosystem Expansion

- Integrating additional modules (e.g., GPS, SD Card storage).

2. Open-Source Readiness

- Finalizing comprehensive GitHub documentation (README, API Wiki) for future researchers.

3. Field Deployment

- Moving from the lab to real-world testing.

Thank You

email: mengxian0913@gmail.com